Scott-Toolkit v6.2.4

Your personal cheat sheet. The toolkit handles context engineering, making sure Claude Code always knows where you left off, what you're building, and what lessons you've learned. It also enforces stack-specific correctness, detects plugin-project misalignment, and feeds lessons back into reusable checks. It works alongside GSD (project management) and Superpowers (dev methodology).

Starting a Session

Just open Claude Code in your project folder. The toolkit handles the rest:

1. **Session-start hook fires automatically** — it checks for context files and shows you what's available
2. **If you see "Resume file found"** — Claude will read `.claude-resume.md` and know exactly where you left off
3. **If you see "No resume file"** — type `/scott:resume-project` for a guided walkthrough of getting back up to speed

If you're in your home directory (`~`), the hook shows your active projects list. Pick one and `cd` into it.

The Slash Commands

These are your workflows. Type them and Claude walks you through each one.

Project Workflows

Command	When to use it
`/scott:new-feature`	Add a feature to the current project with guided workflow
`/scott:new-project`	Start a new project with the guided 8-phase workflow
`/scott:phase-closeout`	Run the mandatory phase closeout (verify, review, reflect, gate)
`/scott:resume`	Pick up where you left off on a project

Stack Enforcement & Learning

Command	When to use it
`/scott:stack-baseline`	First-run stack audit for existing projects
`/scott:stack-review`	Monthly stack health dashboard (check metrics, learning loop)

Learning & Improvement

Command	When to use it

`/scott:compare-sources`	Compare new sources against your toolkit and surface what's actionable
`/scott:update-toolkit`	Update the scott-toolkit with new lessons or patterns

Reference & Knowledge

Command	When to use it
`/scott:debug`	Debug a problem with structured diagnosis
`/scott:n8n-reference`	Complete n8n workflow reference
`/scott:surrealdb`	SurrealDB query patterns, schema design, AI agent architecture

Tools & Utilities

Command	When to use it
`/scott:pause`	Generate a resume prompt for picking up later
`/scott:toolkit-briefing`	Deep-read the toolkit so Claude fully understands it

Business Context (Advosy)

Command	When to use it
`/advosy:context`	Get Advosy company context (leadership, subsidiaries, contacts)
`/advosy:claimsforce`	Get Claimsforce automation context (workflows, placeholders, webhooks)
`/advosy:crm`	CRM design system, mockup patterns, and page prototyping context

You don't need to memorize these. Just describe what you want to do and Claude will suggest the right one.

What Happens Automatically

When you start a session

- Hook scans for project context files (CLAUDE.md, PRD.md, todo.md, resume file)
- Hook rebuilds `~/Sites/Global/ACTIVE-PROJECTS.md` from all your project resume files
- Hook checks plugin-project alignment (warns if Vercel plugin is active on a non-Vercel project)
- You see a short summary of what's available

When context is about to compact

- Hook saves a mechanical backup (`.context-snapshot.md`)
- Hook tells Claude to write a proper resume file (`.claude-resume.md`)
- Hook prompts Claude to capture any notable session patterns to `~/.claude/instinct-candidates.md`
- **You might see Claude write a file right before compaction — that's normal and intentional**

- **After compaction**, Claude re-reads the resume file, snapshot, current task, and any offloaded files

When a session ends

- Hook reminds Claude to write the resume file and update docs
- Hook prompts Claude to capture any notable session patterns
- This is your safety net — even if you forget, the reminder fires

Guard hooks (always running)

- **git push** — Claude only pushes when you explicitly ask; Claude Code's permission prompt is the safety net
- **Destructive commands blocked** — `rm -rf`, `git reset --hard`, etc. require explanation
- **CLAUDE.md protected** — can't be overwritten without confirmation
- **npm install blocked** — new dependencies need your approval
- **SurrealDB syntax validation** — every .surql file write is validated against the live SurrealDB instance
- **SurrealDB Context7 reminder** — writing .ts files with surql imports triggers a reminder to verify against docs
- **SurrealDB integration test advisory** — running tests without live integration tests triggers a warning
- **SurrealDB integration test gate** — phase completion is BLOCKED if the phase touched SurrealDB files but no live integration tests exist

---

Pausing Work

Option A: Just close Claude Code

The session-end hook will remind Claude to write the resume file. If Claude has time before the session closes, it writes `.claude-resume.md` with everything needed to continue.

Option B: Tell Claude "let's stop here"

Claude will: (1) Write/update `.claude-resume.md` with current state, (2) Update CLAUDE.md's Current Status section, (3) Mark completed tasks in `tasks/todo.md`.

> **Option B is better** because Claude has full context when you explicitly pause. If you just close the window, the hook fires but Claude might not have time to respond.

If context fills up mid-session

The PreCompact hook fires automatically. Claude writes the resume file before compaction compresses the conversation. After compaction, Claude reads the resume file back and continues. You shouldn't notice any disruption.

The Resume File (`.claude-resume.md`)

This is the magic. It's a ~10-line file in your project root that tells the next session exactly where to pick up.

You never write this file. Claude writes it. It contains: what workflow/phase you were in, what's done and what's next, and key decisions made during the session.

You never read this file manually either. Claude reads it at session start. It's a machine-to-machine handoff note.

Quick Reference: What Lives Where

Location	What's there	Who maintains it
`~/Sites/Global/scott-toolkit/`	The toolkit repo (source of truth)	You + Claude
`~/.claude/hooks/`	Deployed hooks (symlinked to toolkit)	`setup.sh`
`~/.claude/rules/`	Behavior rules (symlinked to toolkit)	`setup.sh`
`~/.claude/skills/scott-*/`	Deployed workflow skills	`setup.sh`
`~/.claude/checks/`	Stack enforcement check files	`setup.sh`
`~/.claude/tools/`	CLI tools (stack-detect, stack-check, etc.)	`setup.sh`
`~/.claude/settings.json`	Hook registrations, global plugin toggles	Manual
`<project>/stack-lock.json`	Locked technology versions	`/scott:new-project`
`<project>/.claude-resume.md`	Resume file for that project	Claude (auto)

Symlinks (why updates "just work")

The hooks and rules in `~/.claude/` are symlinks pointing back to `~/Sites/Global/scott-toolkit/`. When you `git pull` toolkit updates, the symlinks still point to the same files, so updates take effect instantly. No re-deploy needed unless brand new files were added.

> **Exception:** The pretooluse-router runs as a bundled CJS file (`pretooluse-router.cjs`) under Node instead of Bun, because Bun has a stdin race condition with Claude Code. The TS source files are still the source of truth, but after editing them you must rebuild:
> `bun build hooks/pretooluse-router.ts --outfile hooks/pretooluse-router.cjs --target node --format cjs`

Hooks That Fire on Their Own

Hook	When it fires	What it does
session-start	Every time you open Claude Code	Scans for resume files, rebuilds active projects list, checks plugin alignment
pre-compact	Context window about to compress	Saves backup, tells Claude to write resume file
session-end	Claude Code closing	Reminds Claude to write resume file + update docs
pretooluse-router	Every Bash command	Routes to guard hooks (destructive, npm, phase completion, SurrealDB). Runs as Node CJS.
guard-destructive	`rm -rf`, `git reset --hard`, etc.	Blocks and explains why
guard-claude-md	Edit to CLAUDE.md or MEMORY.md	Blocks until you confirm
guard-npm-install	npm/pnpm install commands	Blocks and lists the packages
guard-phase-completion	GSD marks phase complete	Blocks without closeout marker
auto-format	After Write/Edit on .js/.ts/.vue/.css/.json	Runs Prettier if configured
context-reminders	After every tool use	Warns at 60 min and 100 tool uses
offload-large-output	After every tool use	Writes tool results >4KB to overflow dir
version-propagate	CHANGELOG.md edited	Checks files for stale version references
uiux-reminder	.vue files edited during GSD	Nudge to run /impeccable:audit
extract-instincts	Before compaction + session end	Prompts Claude to note session patterns
pre-completion-checklist	Session ends	Checks for uncommitted changes, stale todo, missing resume
bash logger	Every Bash command	Logs to `~/.claude/bash-commands.log` (silent)

Behavior Rules (Always Loaded)

Rule	What it tells Claude
claude-behavior.md	Operation resolution via interfaces.json. Stack enforcement. Superpowers for dev methodology, GSD for project management. Pre-completion verification. Task contracts. Doom-loop detection. Named agent roles. Checkpoint commits. Context rot monitoring. Post-compaction recovery.
code-style.md	TypeScript strict mode, Vue 3 Composition API, Tailwind v4, Pinia, 2-space indent, single quotes.

PM Mode: GSD + Superpowers

All projects use **GSD + Superpowers** together with distinct roles:

GSD = orchestration engine (what to do, when, and tracking progress)

- Phase planning, execution, verification, state tracking, quick tasks, test coverage

Superpowers = discipline layer (how to do it well)

- Git isolation (worktrees), TDD enforcement, code review, plan quality, branch completion

Operation names (like `plan_phase`, `tdd`) resolve to concrete commands via `config/interfaces.json`.

The build loop

1. Worktree → isolation for GSD execution
2. Plan → structured task breakdown
3. Execute → TDD discipline applies, stack-check runs on changed files
4. Review → after GSD execution completes
5. Verify → UAT against acceptance criteria
6. Finish branch → merge or PR

Phase Auto-Advancement

Tag	Meaning	What Claude Does
`[STOP]`	Judgment phase — needs your input	Pauses and waits for you
`[AUTO]`	Mechanical phase — deterministic	Shows one-line summary, proceeds
`[DELEGATE]`	Hands off to GSD/Superpowers	Shows what was delegated, proceeds

No tag defaults to `[STOP]` (safe default). AUTO means "don't wait for permission," not "skip."

Native Claude Code Commands

Session Management

Command	What it does
`/clear`	Clear conversation and free context
`/compact`	Compress conversation
`/resume`	Resume a previous session
`/fork`	Fork conversation at this point

`/rewind`	Rewind code and/or conversation (`Esc+Esc`)

Code & Git

Command	What it does
`/diff`	Interactive diff viewer
`/review`	Review a PR for quality, correctness, security
`/plan`	Enter plan mode (read-only analysis, then propose)

Model & Output

Command	What it does
`/model`	Change AI model (sonnet, opus, haiku)
`/fast`	Toggle fast mode (same model, faster output)
`/output-style`	Switch between Default, Explanatory, Learning

Useful Keyboard Shortcuts

Shortcut	What it does
`Esc + Esc`	Rewind/summarize
`Shift+Tab`	Cycle permission modes
`Ctrl+V`	Paste image from clipboard
`Ctrl+T`	Toggle task list
`@`	File path autocomplete
`!`	Bash mode (run command directly)